Documentation technique — Assistant DevSecOps

1. Vue d'ensemble

L'Assistant DevSecOps centralise les logs, les résultats de tests et les vulnérabilités d'une application, les analyse automatiquement et s'appuie sur l'IA pour expliquer les incidents et proposer des corrections, toujours validées par un humain.

La plateforme repose sur quatre briques fonctionnelles et une brique de pilotage :

Brique	Rôle	Code principal
Logs	Import (fichier, CI, Loki), recherche, regroupement des erreurs similaires, détection de pics	`LogIngestionService`, `LogAnalysisService`, `LogQueryService`, `TimeSeriesAnomalyDetector`
Tests	Import/lancement JUnit, couverture JaCoCo, historique, tests instables, corrélation échec ↔ logs	`TestOrchestrationService`, `TestRunHistoryService`, `TestLogCorrelationService`
Sécurité	Scans OWASP Dependency-Check, Trivy, SonarQube, score de priorité 0-100, suivi des statuts	`VulnerabilityService`, `RiskScoring`, `*ScanService`, `*ReportParser`
IA	Explication d'incident, recommandations à valider, copilote conversationnel	`AiService`, `AiClient`, service Python `ia-service`
Pilotage	Tableau de bord, score de santé, quality gate, audit, utilisateurs, notifications temps réel	`DashboardService`, `QualityGateService`, `AuditService`, `UserService`, `EventService`

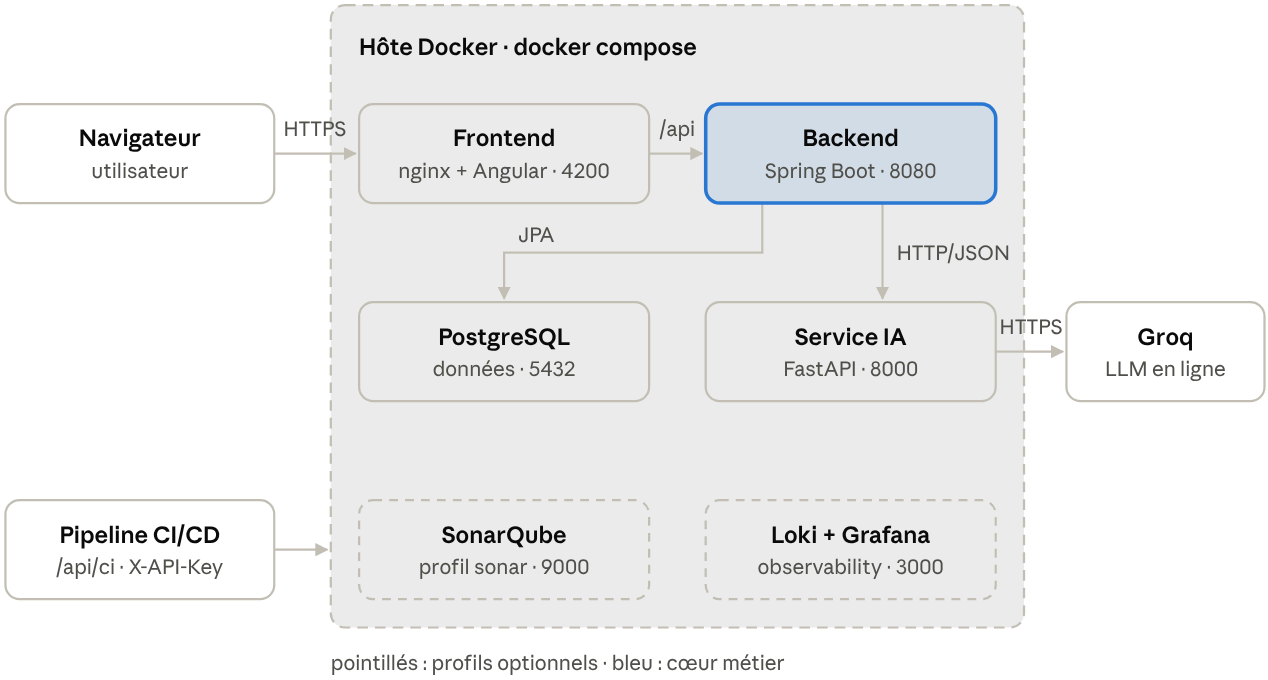
Quatre acteurs utilisent le système : le **Développeur** (consultation, analyse, copilote), le **Responsable qualité** (+ gestion des failles, validation des recommandations, audit), l'**Administrateur** (+ gestion des utilisateurs) et la **chaîne CI/CD**, acteur technique qui publie ses résultats avec une clé d'API.

Arborescence du dépôt :

```
assistant-devsecops/
├── backend/            API Spring Boot 3 (Java 17) : métier, sécurité,
persistance
|   └── src/main/java/com/vermeg/assistant/
|       ├── config/       sécurité, OpenAPI, gestion des erreurs, données
initiales
|       ├── controller/   contrôleurs REST (/api/...)
|       ├── domain/       entités JPA
|       ├── model/        objets métier et DTO (ApiDtos)
|       ├── repository/   dépôts Spring Data
|       ├── security/     JWT, clé d'API CI, rôles, anti-bruteforce
|       └── service/      logique métier (logs, tests, sécurité, IA,
pilotage)
├── ia-service/         microservice Python FastAPI : LLM Groq, ML scikit-
learn, moteur heuristique
├── frontend/           application Angular 22 (standalone components,
signals)
├── infra/              configuration Promtail et Grafana (Loki)
├── samples/            jeux de données de démonstration (logs, tests,
rapports de scan)
├── scripts/            demo-seed.sh, ci/publish-results.sh
├── docker-compose.yml  orchestration de tous les services
├── .github/workflows/ci.yml · .gitlab-ci.yml · Jenkinsfile
└── .env                variables d'environnement (secrets, clés)
```

2. Architecture

Trois services applicatifs indépendants (frontend, backend, service IA) et une base PostgreSQL tournent dans des conteneurs Docker sur un même hôte ; seul le service IA sort vers Internet (API Groq).



architecture physique · 6 conteneurs, 1 service externe

Le navigateur passe par nginx, qui relaie /api au backend ; le pipeline CI parle directement au backend avec sa clé d'API.

Architecture logique en trois couches

1. **Présentation** : application Angular servie par nginx, qui proxifie `/api` vers le backend. Notifications temps réel par Server-Sent Events (`/api/events/stream`).
2. **Métier** : backend Spring Boot. Chaque requête traverse d'abord la chaîne de sécurité (JWT ou clé d'API CI, contrôle des rôles), puis l'un des cinq modules : Logs, Tests, Sécurité, IA, Pilotage.
3. **Données et services** : PostgreSQL (persistance JPA), service IA FastAPI (LLM + ML), outils externes (SonarQube, Trivy, OWASP Dependency-Check, Loki).

Architecture physique (docker-compose)

Service	Image / technologie	Port	Profil	Rôle
frontend	nginx + build Angular	4200 → 80	socle	Interface web, proxy `/api`

Service	Image / technologie	Port	Profil	Rôle
backend	Spring Boot 3, Java 17	8080	socle	API REST, sécurité, métier, Swagger
ia-service	Python 3.12, FastAPI	8000 (interne)	socle	LLM Groq, clustering, détection d'anomalies, heuristique
postgres	postgres:16-alpine	5432 (interne)	socle	Base de données
sonarqube	sonarqube:10-community	9000	`sonar`	Analyse statique du code
loki / promtail / grafana	Grafana 3.1 / 11.2	3100 / – / 3000	`observability`	Centralisation et visualisation des logs

Flux principaux :

- Navigateur → frontend (HTTPS) → backend `/api/**` avec jeton JWT.
- Pipeline CI → backend `/api/ci/**` avec en-tête `X-API-Key`.
- Backend → ia-service en HTTP/JSON (`/explain` , `/chat` , `/ml/cluster` , `/ml/outliers` , `/health`).
- ia-service → Groq (HTTPS + `GROQ_API_KEY`), avec modèles de secours puis moteur heuristique local.
- Backend → socket Docker de l'hôte (lecture seule) pour les scans Trivy d'images locales.

3. Installation et démarrage

Le chemin le plus simple est Docker Compose : une commande démarre la base, le backend, le service IA et l'interface sur http://localhost:4200.

Prérequis : Docker et Docker Compose v2. Pour le développement local : Java 17 + Maven, Node.js 22 + npm, Python 3.12.

Démarrage avec Docker

```
# créer .env à la racine (section 4) ; renseigner au minimum JWT_SECRET (32 caractères min.), GROQ_API_KEY, CI_API_KEY
docker compose up -d --build                    # socle : postgres, backend, ia-service, frontend
docker compose --profile sonar up -d            # + SonarQube (http://localhost:9000)
docker compose --profile observability up -d    # + Loki, Promtail, Grafana (http://localhost:3000)
```

Après une modification du code, reconstruire uniquement le service concerné : `docker compose up -d --build backend` (ou `ia-service`, `frontend`).

Développement local, sans Docker

```
# Backend (base H2 fichier par défaut, aucun serveur à installer)
cd backend && mvn spring-boot:run               # http://localhost:8080, Swagger : /swagger-ui.html

# Service IA
cd ia-service && pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend (/api proxifié vers :8080 par proxy.conf.json)
cd frontend && npm install && npm start         # http://localhost:4200
```

Comptes créés au premier démarrage (base vide uniquement, à changer depuis l'écran Administration) :

Identifiant	Mot de passe par défaut	Rôle
admin	Admin@2026	ADMIN
qualite	Qualite@2026	QUALITY_MANAGER
dev	Dev@2026	DEVELOPER

Données de démonstration : avec `ASSISTANT_DEMO_ENABLED=true`, le backend précharge logs, vulnérabilités et exécutions de tests si la base est vide (fichiers de `backend/src/main/resources/demo/`). Le dossier `samples/` contient des fichiers à importer depuis l'interface. Pendant une démo, `./scripts/demo-seed.sh http://localhost:8080 dev Dev@2026` injecte un incident pour déclencher les notifications temps réel.

4. Configuration

Les secrets et choix d'environnement passent par le fichier `.env` (lu par docker-compose) ; les réglages métier par `backend/src/main/resources/application.yml`, chacun surchargeable par variable d'environnement.

Variables d'environnement

Variable	Service	Obligatoire	Rôle
`JWT_SECRET`	backend	oui	Clé de signature des jetons (32 caractères min.)
`JWT_EXPIRATION_MINUTES`	backend	non (480)	Durée de validité d'une session
`CI_API_KEY`	backend	non	Clé attendue dans l'en-tête `X-API-Key` des pipelines ; vide = désactivée
`DB_URL`, `DB_USERNAME`, `DB_PASSWORD`	backend	non	Base PostgreSQL ; sans `DB_URL`, base H2 fichier `./data`
`BOOTSTRAP_*_PASSWORD`	backend	non	Mots de passe des 3 comptes initiaux
`ASSISTANT_DEMO_ENABLED`	backend	non (true)	Préchargement des données de démo
`AI_SERVICE_URL`	backend	non	URL du service IA (`http://ia-service:8000` en Docker)
`GROQ_API_KEY`	ia-service	non	Clé Groq ; absente = mode heuristique seul
`ASSISTANT_AI_MODEL`	ia-service	non	Modèle principal (`openai/gpt-oss-120b`)
`ASSISTANT_AI_FALLBACK_MODELS`	ia-service	non	Modèles de secours, séparés par des virgules

Variable	Service	Obligatoire	Rôle
`ASSISTANT_AI_REASONING`, `ASSISTANT_AI_CACHE_SIZE`	ia-service	non	Effort de raisonnement (low), taille du cache d'explications (256)
`SONAR_ENABLED`, `SONAR_HOST_URL`, `SONAR_TOKEN`, `SONAR_PROJECT_KEY`	backend	non	Synchronisation SonarQube
`NVD_API_KEY`	backend	non	Accélère la mise à jour de la base OWASP Dependency-Check
`TRIVY_IMAGE`, `DOCKER_GID`	backend	non	Image scannée par Trivy, groupe du socket Docker
`LOKI_ENABLED`, `LOKI_URL`	backend	non	Collecte périodique des logs depuis Loki (30 s)
`LOG_RETENTION_DAYS`	backend	non (30)	Purge automatique des logs anciens
`TZ`	backend	non	Fuseau horaire (Africa/Tunis)

Paramètres métier (`assistant.*` dans application.yml)

Clé	Valeur	Effet
`logs.anomaly-threshold`	3	Occurrences minimales pour former un groupe d'anomalies
`logs.similarity-threshold`	0.6	Similarité pour regrouper deux messages
`logs.spike-bucket-minutes` / `spike-z-threshold` / `spike-min-count`	5 / 2.5 / 3	Fenêtres et sensibilité de la détection de pics
`logs.analysis-window`	20 000	Logs récents analysés
`tests.flaky-window`	10	Exécutions observées pour détecter un test instable
`login.max-attempts` / `lock-minutes`	5 / 5	Verrouillage après échecs de connexion

Clé	Valeur	Effet
`quality-gate.max-critical-vulnerabilities`	0	Seuil de la quality gate
`quality-gate.max-high-vulnerabilities`	5	Seuil de la quality gate
`quality-gate.min-test-success-rate`	95 %	Seuil de la quality gate
`quality-gate.min-line-coverage`	40 %	Seuil de la quality gate (0 = condition désactivée)
`quality-gate.max-critical-anomalies`	10	Seuil de la quality gate
`ai.timeout-seconds`	60	Délai maximal d'un appel au service IA

Le fichier `.env` contient des secrets : il ne doit jamais être commité (il est dans `.gitignore`).

5. Backend Spring Boot

Le backend (`com.vermeg.assistant`, Java 17, Spring Boot 3.3) porte toute la logique métier : il ingère et analyse les données, décide de la quality gate, sécurise l'API et délègue l'IA au microservice Python.

Organisation en couches : `controller` (REST) → `service` (métier) → `repository` (Spring Data JPA) → `domain` (entités). Les objets échangés sont dans `model` (objets métier + records `ApiDtos`).

Brique Logs

Classe	Responsabilité
`LogIngestionService`	Point d'entrée unique (API, CI, upload, Loki) : parsing, persistance, notification des nouvelles anomalies critiques
`LogParserService`	Parseur multi-formats : classique `date NIVEAU [source] message`, Spring Boot 3, JSON/NDJSON

Classe	Responsabilité
`LogStore`	Accès aux logs persistés (PostgreSQL / H2)
`LogAnalysisService`	Normalise les messages (nombres, UUID, IP, chemins), regroupe les identiques puis fusionne les proches (`TextSimilarityUtil`)
`AnomalyCategorizer` / `AnomalySummaryService`	Catégorie (DATABASE, NETWORK, SECURITY…) et résumé en langage naturel par règles
`TimeSeriesAnomalyDetector`	Pics de fréquence : comptage par fenêtre de 5 min, z-score robuste
`LogQueryService`	Recherche paginée, statistiques, pics (z-score local + Isolation Forest via le service IA)
`LokiLogPuller`	Collecte périodique depuis Grafana Loki (optionnelle)

Brique Tests

Classe	Responsabilité
`TestOrchestrationService`	Lancement asynchrone, import des rapports CI, résultats consolidés
`TestExecutionService`	Exécute `mvn test` sur le projet surveillé et mémorise la fenêtre temporelle du run
`SurefireReportParser` / `JacocoReportParser`	Lecture des rapports JUnit XML et de la couverture JaCoCo
`TestRunHistoryService`	Historique, tendance, détection des tests instables (fenêtre de 10 runs)
`TestLogCorrelationService`	Relie chaque échec aux logs émis pendant le run (fenêtre temporelle, puis similarité)

Brique Sécurité

Classe	Responsabilité
`VulnerabilityService`	Consolidation toutes sources : création (NEW), mise à jour, résolution automatique, réouverture ; statistiques
`RiskScoring`	Score de priorité 0-100 : gravité/CVSS (jusqu'à 80 pts) + exposition (+5) + correctif disponible (+8) + ancienneté (jusqu'à +7)
`DependencyScanService` / `DependencyCheckReportParser`	OWASP Dependency-Check sur les bibliothèques Java
`TrivyScanService` / `TrivyReportMapper`	Scan d'images de conteneurs, version corrective
`SonarIssueFetcher` / `SonarIssueMapper`	Synchronisation des vulnérabilités SonarQube (toutes les 10 min)
`SecurityScanService`	Lance les scans sous forme de jobs

Brique IA

Classe	Responsabilité
`AiService`	Explication d'incident (cache par incident, régénération), validation humaine, copilote ; `buildContext()` construit l'instantané du projet (12 000 caractères max.)
`AiClient`	Client HTTP du service IA ; le backend ne connaît que le contrat JSON
`AiRecommendationStore`	Recommandations persistées, toujours créées en PENDING_REVIEW

Pilotage et transverse

Classe	Responsabilité
`DashboardService`	Vue consolidée et score de santé : 100 − pénalités (vulnérabilités ⩽ 40, tests ⩽ 25, anomalies critiques ⩽ 20, pics ⩽ 10)
`QualityGateService`	Décision PASSED/FAILED selon les seuils `quality-gate.*`
`JobService` / `ProcessRunner`	Traitements longs asynchrones (Maven, Trivy, Sonar) avec timeout ; l'API répond immédiatement avec un identifiant de job
`EventService`	Notifications Server-Sent Events (anomalie critique, vulnérabilité critique, fin de job), battement toutes les 25 s
`AuditService`	Journal d'audit des actions sensibles
`ReportService`	Exports CSV des vulnérabilités et synthèse Markdown
`ToolStatusService`	Indique avant le clic si Maven, Trivy, Docker, SonarQube et les clés sont disponibles
`MaintenanceScheduler`	Purge des logs anciens (2 h 30 chaque nuit), synchronisation Sonar
`UserService`	Connexion, gestion des comptes, changement de mot de passe

Sécurité de l'API (`config/SecurityConfig` , package `security`)

- API sans état : jeton JWT dans `Authorization: Bearer …` (`JwtAuthenticationFilter` , `JwtService`), CSRF désactivé, mots de passe hachés (BCrypt).
- Le flux SSE accepte le jeton en paramètre `token` , car `EventSource` ne permet pas d'en-tête.
- Pipeline CI : en-tête `X-API-Key` comparé en temps constant (`ApiKeyAuthenticationFilter`), rôle `CI` , routes `/api/ci/**` uniquement.
- Rôles (`Role`) : `DEVELOPER` , `QUALITY_MANAGER` , `ADMIN` , `CI` . Autorisations par route et par méthode (`@PreAuthorize`).

- Anti-bruteforce : 5 échecs → compte verrouillé 5 min (`LoginAttemptService`).
- Erreurs homogènes en JSON (`GlobalExceptionHandler`) ; service IA injoignable → HTTP 502.

Démarrage : `DataInitializer` crée les comptes par défaut et, en mode démo, les données d'exemple. `OpenApiConfig` expose la documentation Swagger sur `/swagger-ui.html` .

6. Référence de l'API REST

Toutes les routes sont sous `/api` et documentées en direct dans Swagger (`http://localhost:8080/swagger-ui.html`). Sauf mention contraire, elles sont ouvertes aux trois rôles humains (DEV, QUALITY_MANAGER, ADMIN) ; « Resp. » = QUALITY_MANAGER ou ADMIN.

Méthode	Route	Rôle	Description
POST	`/auth/login`	public	Connexion, renvoie `{token, expiresAt, user}`
GET	`/auth/me`	connecté	Utilisateur courant
POST	`/auth/change-password`	connecté	Changement de mot de passe
GET	`/dashboard/overview`	tous	Score de santé, indicateurs, services, jobs récents
GET	`/dashboard/quality-gate`	tous	Résultat de la quality gate
GET	`/logs` , `/logs/search`	tous	Logs ; recherche paginée (niveau, source, texte, période)
GET	`/logs/stats` , `/logs/sources`	tous	Statistiques, liste des services
GET	`/logs/anomalies?engine=local`	tous	Groupes d'anomalies

Méthode	Route	Rôle	Description
GET	`/logs/spikes`	tous	Pics de fréquence d'erreurs
GET	`/logs/collectors`	tous	État des collecteurs (Loki)
POST	`/logs/ingest?source=`	tous	Ingestion de texte brut, JSON ou NDJSON
POST	`/logs/upload`	tous	Import d'un fichier de logs (multipart `file`)
DELETE	`/logs`	ADMIN	Vider les logs
POST	`/tests/run`	tous	Lance `mvn test` (job asynchrone)
POST	`/tests/import`	tous	Import de rapports JUnit/JaCoCo (multipart `reports`)
GET	`/tests/results`, `/tests/runs/{id}`	tous	Dernière exécution, exécution donnée
GET	`/tests/history`, `/tests/flaky`	tous	Historique, tests instables
GET	`/tests/failures/correlated`	tous	Échecs corrélés aux logs
GET	`/security/vulnerabilities`	tous	Liste filtrable (status, severity, source, q)
GET	`/security/vulnerabilities/{recordId}`, `/security/stats`	tous	Détail, statistiques
PATCH	`/security/vulnerabilities/{recordId}`	Resp.	Statut, assigné, note
POST	`/security/scan/dependencies`, `/scan/images`, `/scan/sonar`	Resp.	Lance un scan (job)
POST	`/security/import/{type}`	Resp.	Import d'un rapport `dependency-check` ou `trivy`

Méthode	Route	Rôle	Description
POST	`/ai/explain`	tous	Explication d'un incident → recommandation PENDING_REVIEW
GET	`/ai/recommendations`, `/ai/recommendations/{id}`	tous	Liste (filtre `status`), détail
PATCH	`/ai/recommendations/{id}/status`	tous	Validation humaine `APPROVED` / `REJECTED` + commentaire
POST	`/ai/chat`	tous	Copilote : `{question, history}` → `{answer, provider, model}`
GET	`/ai/health`	tous	État et fournisseur du service IA
GET	`/jobs`, `/jobs/running`, `/jobs/{id}`	tous	Suivi des traitements longs
GET	`/events/stream?token=`	tous	Flux temps réel SSE
GET	`/reports/vulnerabilities.csv`, `/reports/summary.md`	tous	Exports
GET	`/system/tools`	tous	Disponibilité des outils (Maven, Trivy, Docker, Sonar)
GET	`/audit?limit=200`	Resp.	Journal d'audit
GET/POST/PUT/DELETE	`/users`, `/users/{id}`	ADMIN	Gestion des utilisateurs
POST	`/ci/logs`, `/ci/tests`, `/ci/security/{type}`	CI (X-API-Key)	Publication des résultats par le pipeline
GET	`/ci/quality-gate`	CI	Quality gate : HTTP 200 si PASSED, 412 si FAILED

Exemple : connexion puis question au copilote.

```
TOKEN=$(curl -s -H 'Content-Type: application/json' \
  -d '{"username":"dev","password":"Dev@2026"}'
http://localhost:8080/api/auth/login | jq -r .token)

curl -s -H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json'
\
  -d '{"question":"Quelles vulnérabilités critiques corriger en premier
?","history":[]}' \
  http://localhost:8080/api/ai/chat
```

7. Service IA (FastAPI)

Le microservice `ia-service` (Python 3.12) concentre tout ce qui relève de l'IA : il répond toujours, grâce à une cascade LLM principal → LLM de secours → moteur heuristique local.

Modules

Fichier	Rôle
`app/main.py`	Application FastAPI, routes, bascule vers l'heuristique en cas d'échec du LLM
`app/llm.py`	Client Groq (import différé), cascade de modèles pour le chat, parsing JSON défensif, cache LRU des explications
`app/prompts.py`	Prompts système : explication d'incident (réponse JSON stricte) et copilote (règles + guide de la plateforme + instantané)
`app/heuristics.py`	Moteur local sans LLM : 13 règles d'explication et copilote par intentions
`app/ml.py`	Clustering TF-IDF + DBSCAN, détection d'anomalies Isolation Forest
`app/schemas.py`	Contrats Pydantic (requêtes et réponses)

Routes

Route	Entrée	Sortie
`POST /explain`	type d'incident, source, message, occurrences, contexte, `regenerate`	explication, correctif, gravité, fournisseur, modèle
`POST /chat`	question, historique (12 derniers tours), instantané du projet (20 000 caractères max.)	réponse, fournisseur, modèle réellement utilisé
`POST /ml/cluster`	messages, `eps`	étiquette de groupe par message, nombre de groupes
`POST /ml/outliers`	fenêtres + caractéristiques numériques	fenêtres atypiques
`GET /health`	–	statut, fournisseur actif, modèle, clé configurée

Explication d'incident : le prompt impose une réponse JSON `{explanation, suggested_fix, severity_assessment}`. `_parse_json_response` tolère du texte autour du JSON. Les réponses sont mises en cache (clé SHA-256 du prompt) ; `regenerate=true` ignore le cache et monte la température à 0.8. Sans LLM, `heuristics.explain` reconnaît 13 motifs : Log4Shell, Text4Shell, pool de connexions saturé, timeout base, deadlock, mémoire insuffisante, JWT, SSL, service amont, NullPointerException, assertion de test, disque plein, configuration.

Copilote

1. Le backend construit l'instantané du projet (`AiService.buildContext`) : synthèse (score de santé, quality gate), logs, anomalies, tests, historique, tests instables, statistiques et liste des vulnérabilités, recommandations, activité, traitements.
2. `prompts.build_chat_messages` assemble : prompt système + instantané, 12 derniers échanges, question.
3. `llm.chat` essaie `ASSISTANT_AI_MODEL` (par défaut `openai/gpt-oss-120b`) puis chaque modèle de `ASSISTANT_AI_FALLBACK_MODELS` (`llama-3.3-70b-versatile` , `llama-3.1-8b-instant`) ; un quota dépassé, une erreur ou une réponse vide fait passer au suivant.

4. Si tous échouent, `heuristics.chat` répond : salutations sans données, définitions de 14 notions, rubrique demandée (santé, logs, tests, sécurité, recommandations, activité…), relance sur le sujet précédent, recherche libre des mots de la question dans l'instantané.

Apprentissage automatique (`ml.py`)

- *Regroupement des erreurs* : deux vectorisations TF-IDF (mots + n-grammes de 3 à 5 caractères), puis DBSCAN en distance cosinus (`min_samples=2`). Des milliers de lignes deviennent quelques familles.
- *Fenêtres atypiques* : Isolation Forest (200 arbres, `contamination=auto` , graine fixe), appelé par `LogQueryService` en complément du z-score robuste calculé côté Java.

Dépendances : fastapi, uvicorn, pydantic 2, groq, scikit-learn, numpy, scipy. Le conteneur tourne sous un utilisateur non root avec un healthcheck sur `/health` .

8. Frontend Angular

L'interface est une application Angular 22 en composants autonomes (standalone), basée sur les signals et le nouveau control flow `@if` / `@for` , sans bibliothèque UI externe : les graphiques sont des composants SVG maison.

Structure (`frontend/src/app`)

Dossier	Contenu
`core/`	`models.ts` (contrats alignés sur les DTO Java), `api.service.ts` (tous les appels HTTP), `auth.service.ts` , `auth.guard.ts` , `auth.interceptor.ts` , `events.service.ts` (SSE), `toast.service.ts` , `ai-chat.store.ts` (état du copilote), `format.ts`
`shared/`	`charts.ts` (barres, courbes, anneau), `pipes.ts` , `explain-dialog.component.ts` (fenêtre « Expliquer (IA) »), `tool-status.component.ts` , `toast-container.component.ts`
`features/`	Un dossier par écran : `login` , `shell` (menu, notifications), `dashboard` , `logs` , `tests` , `security` , `ai` , `admin`

Routes (`app.routes.ts` , chargement différé de chaque écran)

Chemin	Écran	Accès
`/login`	Connexion	public
`/dashboard`	Tableau de bord : score de santé, quality gate, indicateurs, jobs	connecté
`/logs`	Recherche, statistiques, anomalies, pics, import	connecté
`/tests`	Résultats, historique, tests instables, corrélations, lancement/import	connecté
`/security`	Vulnérabilités, statistiques, scans, import de rapports	connecté (actions : Resp.)
`/ai`	Copilote et recommandations	connecté
`/admin`	Utilisateurs, journal d'audit	ADMIN, QUALITY_MANAGER

Authentification : `AuthService` stocke `{token, expiresAt, user}` dans le `sessionStorage` (effacé à la fermeture de l'onglet) et vérifie l'expiration côté client. `authInterceptor` ajoute `Authorization: Bearer` et déconnecte sur HTTP 401. `authGuard` et `roleGuard` protègent les routes ; le backend applique de toute façon les mêmes règles.

Temps réel : `EventsService` ouvre un `EventSource` sur `/api/events/stream?token=…` après la connexion. Chaque événement (`anomaly.critical` , `vulnerability.critical` , `job.finished` , `tests.completed` , `logs.ingested`) est diffusé aux écrans ; tous sauf logs.ingested alimentent aussi la cloche de notifications et un toast.

Écran Assistant IA (`features/ai`)

- La conversation vit dans `AiChatStore` (service singleton), pas dans le composant : elle survit aux changements d'écran.
- Elle est sauvegardée dans le `localStorage` sous la clé `assistant.copilot.<utilisateur>` (200 messages max.) : retrouvée après déconnexion ou rechargement, invisible pour un autre compte. L'onglet, le filtre et le brouillon sont aussi conservés.

- Une réponse qui arrive après avoir quitté l'écran (ou après déconnexion) est rangée dans la bonne conversation. Seuls les 20 derniers messages sont renvoyés comme contexte.
- Onglet Recommandations : filtre « Toutes » par défaut, fiches à valider en premier, rafraîchissement automatique sur événement serveur et après chaque validation.

Commandes

```
npm start        # serveur de développement, /api proxifié vers :8080
npm test         # tests unitaires (Vitest)
npm run build    # build de production dans dist/frontend/browser
```

En production, l'image Docker sert le build via nginx, qui relaie `/api` vers le backend sans mise en tampon pour le flux SSE.

9. Modèle de données

Six entités JPA (`backend/.../domain`) sont persistées dans PostgreSQL (H2 en développement) ; le schéma est créé et mis à jour automatiquement par Hibernate (`ddl-auto: update`).

Table	Entité	Champs principaux	Index
`app_user`	`AppUser`	username (unique), passwordHash (BCrypt), fullName, email, role, enabled, createdAt, lastLoginAt	–
`log_record`	`LogRecord`	timestamp, level, source, message, stackTrace, ingestedAt	timestamp, level, source
`test_run`	`TestRunRecord`	executedAt, fenêtre début/fin, trigger, branch, commitSha, pipelineId, totaux (tests, failures, errors, skipped), successRate, lineCoverage, branchCoverage, durationSeconds, suitesJson	–

Table	Entité	Champs principaux	Index
`vulnerability_record`	`VulnerabilityRecord`	source, vulnId (CVE), component, severity, cvssScore, description, fixedVersion, status, dates de détection/résolution, autoResolved, reopened, assignee, note	status
`ai_recommendation`	`AiRecommendationRecord`	id (UUID), incidentType, incidentSummary, incidentRef, source, explanation, suggestedFix, severityAssessment, status, provider, model, createdBy, reviewedBy, reviewedAt, reviewComment	incident_ref
`audit_event`	`AuditEvent`	at, username, action, target, details	event_at

Énumérations

- `Role` : DEVELOPER, QUALITY_MANAGER, ADMIN, CI.
- `LogLevel` : DEBUG, INFO, WARN, ERROR, FATAL, UNKNOWN.
- `Severity` : CRITICAL, HIGH, MEDIUM, LOW, UNKNOWN.
- Statut de vulnérabilité : NEW, IN_PROGRESS, FIXED, ACCEPTED_RISK, FALSE_POSITIVE.
- Statut de recommandation : PENDING_REVIEW, APPROVED, REJECTED.

Relations logiques (sans clé étrangère, les liens sont des valeurs) : une recommandation explique un incident identifié par `incidentRef` (groupe de logs, test ou vulnérabilité) ; un événement d'audit et une validation référencent l'utilisateur par son `username` ; un échec de test est rapproché des logs par sa fenêtre temporelle. Les groupes d'anomalies, les pics et les corrélations sont calculés à la volée et ne sont pas stockés.

Le diagramme de classes UML de la conception figure dans la présentation (annexe).

10. Intégration CI/CD

Les trois pipelines fournis (GitHub Actions, GitLab CI, Jenkins) testent et analysent le projet, publient leurs résultats dans l'assistant, puis échouent si la quality gate n'est pas franchie.

Étapes communes

1. Tests backend et couverture : `mvn verify -Dmaven.test.failure.ignore=true` (les échecs sont remontés, la décision revient à la quality gate).
2. Tests du service IA : `pytest` avec rapport JUnit.
3. Frontend : `npm ci`, `npm test`, `npm run build`.
4. Sécurité : OWASP Dependency-Check (JSON), SonarQube si un jeton est configuré.
5. Construction des images Docker et scan Trivy.
6. Publication et quality gate : `scripts/ci/publish-results.sh`.

Fichier	Outil	Organisation
`.github/workflows/ci.yml`	GitHub Actions	Jobs backend, ia-service, frontend, images ; artefacts de rapports ; images poussées sur ghcr.io
`.gitlab-ci.yml`	GitLab CI	Stages test → security → package → publish
`Jenkinsfile`	Jenkins	Tests backend, tests IA, build frontend, Dependency-Check, image + Trivy, publication + quality gate

Script `publish-results.sh`

- Variables requises : `ASSISTANT_URL`, `ASSISTANT_API_KEY` (= `CI_API_KEY` du backend).
- Variables optionnelles : `BRANCH`, `COMMIT_SHA`, `PIPELINE_ID`, `APP_LOG_FILE`, `SUREFIRE_DIR`, `JACOCO_XML`, `DEPCHECK_JSON`, `TRIVY_JSON`, `ENFORCE_GATE` (true par défaut).
- Envoie les rapports Surefire + JaCoCo à `/api/ci/tests`, les rapports de sécurité à `/api/ci/security/{dependency-check|trivy}`, les logs à `/api/ci/logs`.
- Interroge enfin `/api/ci/quality-gate` : HTTP 412 (FAILED) fait échouer le job si `ENFORCE_GATE=true`.

Secrets à déclarer dans l'outil de CI : `ASSISTANT_URL`, `ASSISTANT_API_KEY`, et selon les besoins `NVD_API_KEY`, `SONAR_TOKEN`, `SONAR_HOST_URL`.

11. Tests automatisés

Le projet compte 72 tests automatisés : 45 côté backend (JUnit 5), 24 côté service IA (pytest) et 3 côté frontend (Vitest).

Composant	Fichier de test	Tests	Ce qui est vérifié
Backend	`ApiSecurityIntegrationTests`	12	Contexte Spring complet : connexion, jeton falsifié, droits par rôle, clé d'API CI, ingestion, IA indisponible → 502
Backend	`LogAnalysisTests`	7	Normalisation et regroupement des messages, anomalies
Backend	`ParsersAndDetectorsTests`	5	Parseurs de logs et détection de pics
Backend	`AiRecommendationStoreTests`	4	Cycle de vie des recommandations (PENDING → APPROVED/REJECTED)
Backend	`DependencyCheckReportParserTests`, `TrivyReportMapperTests`, `SonarIssueMapperTests`	3 + 3 + 3	Lecture des rapports de sécurité
Backend	`VulnerabilityServiceTests`	3	Création, résolution automatique, réouverture
Backend	`TestLogCorrelationTests`, `SurefireReportParserTests`	3 + 2	Corrélation échecs ↔ logs, rapports JUnit
Service IA	`test_heuristics.py`	9	Règles d'explication ; copilote : salutations sans données, rubriques, définitions, relances
Service IA	`test_api.py`	6	Routes FastAPI en mode heuristique (sans clé Groq)

Composant	Fichier de test	Tests	Ce qui est vérifié
Service IA	`test_ia_service.py`	5	Construction des prompts, parsing JSON défensif des réponses du LLM
Service IA	`test_ml.py`	4	Clustering DBSCAN et Isolation Forest
Frontend	`core/format.spec.ts`	3	Fonctions de formatage

Lancer les tests

```
cd backend && mvn test                       # profil test : base H2 en mémoire, démo désactivée
cd backend && mvn verify                     # + rapport de couverture JaCoCo (target/site/jacoco)
cd ia-service && pip install -r requirements-dev.txt && pytest -q
cd frontend && npm test
```

Les tests backend utilisent `src/test/resources/application-test.yml` (H2 en mémoire, `ddl-auto: create-drop`, clé CI `test-ci-key`) : ils ne nécessitent ni PostgreSQL ni service IA.

12. Dépannage, limites et évolutions

La plupart des incidents viennent d'une variable d'environnement manquante ou d'un outil externe absent ; l'écran Tests/Sécurité (`/api/system/tools`) indique d'ailleurs ce qui manque avant chaque lancement.

Symptôme	Cause probable	Correction
docker compose refuse de démarrer	`JWT_SECRET` absent ou trop court	Le définir dans `.env` (32 caractères min.)
Copilote en « Mode heuristique »	Clé Groq absente, quota dépassé ou modèles indisponibles	Vérifier `GROQ_API_KEY` ; lire les logs de `ia-service` (« Copilote : échec avec … »)

Symptôme	Cause probable	Correction
HTTP 502 sur `/api/ai/*`	Service IA injoignable	`docker compose ps`, puis `docker compose logs ia-service`
Compte verrouillé	5 échecs de connexion	Attendre 5 minutes
Scan Trivy en échec	Pas d'accès au socket Docker	Régler `DOCKER_GID` (Linux) ou `DOCKER_GID=0` (Docker Desktop)
Dependency-Check très long	Premier téléchargement de la base NVD	Renseigner `NVD_API_KEY`
Horaires décalés d'une heure	Conteneur en UTC	`TZ=Africa/Tunis`
Pipeline CI refusé (401)	Clé différente de `CI_API_KEY`	Aligner `ASSISTANT_API_KEY` du pipeline sur `CI_API_KEY`
Conversation du copilote absente	Autre navigateur ou données du site effacées	Elle est stockée localement par navigateur et par utilisateur

Limites connues

- L'instantané envoyé au copilote est borné à 12 000 caractères : les listes sont tronquées (15 anomalies, 20 vulnérabilités…), le copilote renvoie alors vers l'écran concerné.
- L'historique du copilote est stocké dans le navigateur, pas en base : il ne suit pas l'utilisateur d'un poste à l'autre.
- Le déploiement cible un hôte unique (Docker Compose), sans haute disponibilité.
- Les quotas gratuits de Groq limitent le nombre de requêtes par minute.

Évolutions envisagées

- Passage à l'échelle sur Kubernetes.
- IA enrichie par la documentation interne de Vermeg (RAG).
- Correctifs proposés directement sous forme de merge request.
- Prédiction des incidents avant qu'ils ne surviennent.
- Persistance de l'historique du copilote côté serveur.